Ciência da Computação

Programação Orientada a Objetos

Prof. Luciano Rodrigo Ferretto

IntelliJ IDEA

- AVA – Seção – Preparação do Ambiente
- https://www.youtube.com/watch?v=iJxLUQjKsFc

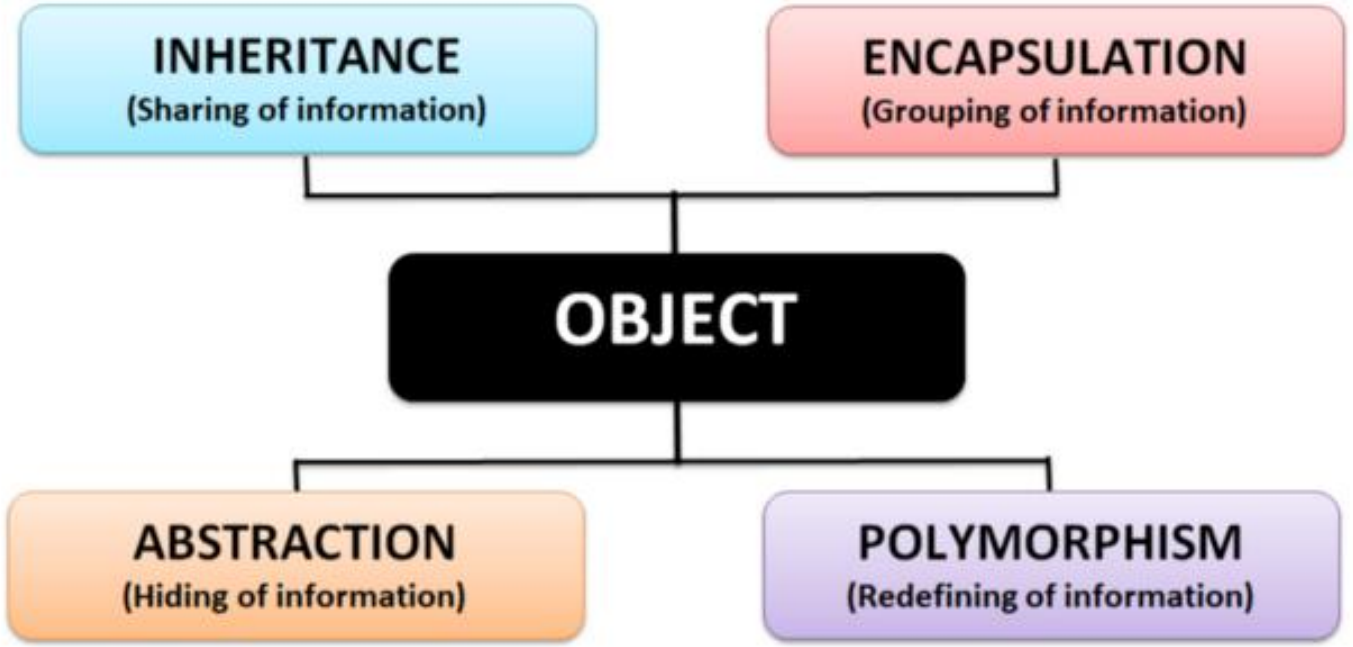


Uma breve revisão...

Abstração, Encapsulamento, Herança, Polimorfismo, Classe Object

Pilares da Orientação a objetos





Ciência da Computação

Abstração

- **Abstração** é o processo de identificar e definir características essenciais de objetos do mundo real, representando essas características em forma de **classes** e **objetos**.

- Uma **classe** é uma estrutura que define atributos e métodos representando um tipo de objeto, fornecendo um modelo para criar instâncias desse tipo.
- Um **objeto** é uma instância de uma classe, caracterizado por seus atributos e comportamentos definidos na classe.
  - Ele pode interagir com outros objetos por meio de métodos e troca de dados.

Objeto é uma **instância** concreta de uma classe na POO.

Ciência da Computação

Encapsulamento

- **Encapsulamento** é a prática de ocultar os detalhes internos de um objeto, expondo apenas as operações relevantes para manipular esses detalhes. Isso promove a modularidade, a segurança e a manutenibilidade do código.

- Na maioria das linguagens de programação orientada a objetos, o encapsulamento é alcançado através da definição de **modificadores de acesso.**
  - **public, private**, **protected e default** (sem modificador)

Ciência da Computação

Herança

- **Herança** é o princípio que permite que uma classe (subclasse) herde os atributos e métodos de outra classe (superclasse), facilitando a reutilização de código, a organização hierárquica e a promoção de relações entre objetos.

- Quando há duas ou mais classes com atributos e métodos em comum, a herança facilita a criação de uma estrutura hierárquica. Nessa estrutura, uma classe "pai" define os elementos comuns que serão herdados pelas classes "filhas".

Ciência da Computação

Polimorfismo

- **Polimorfismo** é um princípio da programação orientada a objetos que permite que funções ou métodos assumam comportamentos diferentes, dependendo do contexto.

- Com o polimorfismo, é possível reutilizar e adaptar comportamentos em diferentes contextos, permitindo que classes e métodos sejam usados de maneira flexível, sem perder a consistência da interface ou lógica comum.

- Ele se divide em dois tipos principais:
  - Polimorfismo Estático
  - Polimorfismo Dinâmico

Ciência da Computação

Polimorfismo

- **Polimorfismo dinâmico** (em tempo de execução): Permite que objetos de sub-classes de uma mesma classe base invoquem métodos que têm a mesma assinatura (nome e parâmetros), mas que se comportam de maneira diferente, de acordo com o tipo específico do objeto instanciado.
  - Sobrescrita de métodos

- **Polimorfismo estático** (em tempo de compilação): Refere-se à capacidade de métodos com o mesmo nome, mas com assinaturas diferentes (número ou tipos de parâmetros), serem utilizados de formas distintas.
  - Sobrecarga de métodos.

Ciência da Computação

Polimorfismo - Estático x Dinâmico

Estático	Dinâmico
Também conhecido como **Sobrecarga de Método.**	Também conhecido como **Sobrescrita de Método.**
Ocorre quando há vários métodos com o mesmo nome, mas com diferentes assinaturas (diferentes tipos ou números de parâmetros) dentro da mesma classe.	Ocorre quando uma subclasse fornece uma implementação específica de um método que já é definido na sua superclasse.
A ligação (binding) das chamadas de método ao código correspondente é feita em tempo de compilação.	A ligação (binding) das chamadas de método ao código correspondente é feita em tempo de execução.
Permite que métodos com o mesmo nome (assinaturas diferentes) possam ser chamados, e a versão correta do método é invocada de acordo com os parâmetros fornecidos na chamada	Permite que o mesmo método (mesma assinatura) possa ser chamado em diferentes tipos de objetos, e a versão correta do método é invocada de acordo com o tipo real do objeto em tempo de execução.

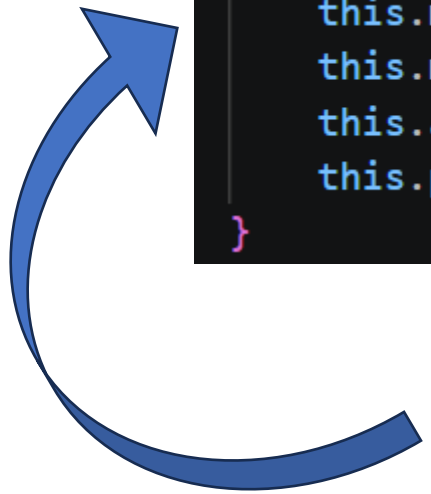
Palavra Chave “Super”

Acesso via “super” - constructors

Usando super para chamar construtores da superclasse

Uma subclasse pode chamar um construtor definido por sua superclasse usando a forma de **super** a seguir:

super(*lista-parâmetros*);



```java
public Veiculo(String _marca, String modelo, int ano, String placa){
    this.marca = _marca;
    this.modelo = modelo;
    this.ano = ano;
    this.placa = placa;
}
```

```java
public Carro(String marca, String modelo, int ano, String placa,
             int quantidadePortas, String tipoCombustivel) {
    super(marca, modelo, ano, placa);
    this.quantidadePortas = quantidadePortas;
    this.tipoCombustivel = tipoCombustivel;
}
```

Ciência da Computação

Acesso via “super”

Usando super para acessar membros da superclasse

Há uma segunda forma de **super** que age um pouco como **this**, exceto por referenciar sempre a superclasse da subclasse em que é usada. Essa aplicação tem a forma geral a seguir:

super.*membro*

Aqui, *membro* pode ser um método ou uma variável de instância.

```
@Override
public String toString() {
    Str g descricao = """
            %s %s (%d)
            Placa: %s
            """;
    return descricao.formatted(
        this.getMarca(),
        this.getModelo(),
        this.getAno(),
        this.getPlaca());
}
```

```
@Override
public String toString() {
    String descricao = super.toString();
    descricao += """
            - Quantidade de portas: %d - Tipo de combustível: %s
            """;
    return descricao.formatted(
            this.getQuantidadePortas(),
            this.getTipoCombustivel());
}
```



- **Livro físico**: possui quantidade de exemplares, localização na biblioteca e outras características físicas.
- **Livro digital**: possui formato de arquivo, tamanho e endereço para acesso ou download.
- **Audiolivro**: possui duração, narrador e informações sobre o arquivo de áudio.

Palavra chave “**abstract**”

Classes e Métodos abstratos

Classes Abstratas

- Uma **classe abstrata** é uma classe que **NÃO pode ser instanciada** diretamente, ou seja, **você não pode criar objetos a partir dela**.
- Em vez disso, ela é projetada para ser uma classe base que fornece uma estrutura comum e recursos compartilhados para suas subclasses.
- Para declarar uma classe abstrata em Java, você utiliza a palavra-chave **abstract** antes da palavra-chave **class**

```
public abstract class Livro {
    private String titulo;
    private String autor;
    private int anoPublicacao;
    private int numeroPaginas;
```

```
Livro novoLivro = new Livro();
```

```
Cannot instantiate the type Livro Java(16777373)
Livro
View Problem (Alt+...  Quick Fix... (Ctrl...  Fix using Copilot (Ctrl...
```

Computação

Classes Abstratas

As classes que não são abstratas e herdam classes abstratas são denominadas classes concretas. Sobre as classes abstratas, podemos destacar os seguintes conceitos.

- Uma classe abstrata serve como modelo para uma classe concreta.
- Como são apenas modelos, não podem ser instanciadas diretamente.
- Por não ser instanciadas, devem ser herdadas por classes concretas.
- Uma classe abstrata pode, ou não, conter métodos abstratos, ou seja, pode ou não implementar um método.
- Contudo, métodos abstratos definidos em uma classe abstrata devem, obrigatoriamente, ser implementados em uma classe concreta.

Fonte: SILVA, F. M.; LEITE, M. C. D.; OLIVEIRA, D. B. **Paradigmas de programação**. Porto Alegre: Sagah, 2019.

Ciência da Computação

Métodos Abstratos

- Uma **classe abstrata** pode conter **métodos abstratos**, que são declarados sem uma implementação.
- Os métodos abstratos servem como "contratos" que as subclasses devem implementar.
- A lógica é dizer o que precisa ser feito, e não como fazer.

```
public abstract class Livro {

    // Método abstrato para definir o formato do livro
    //     (ex.: "Físico" ou "Digital")
    public abstract String getFormato();
```

```
public class LivroDigital extends Livro {
```

```
The type LivroDigital must implement the inherited abstract method
Livro.getFormato() Java(67109264)

LivroDigital

View Problem (Alt+F8)   Quick Fix... (Ctrl+.)   Fix using Copilot (Ctrl+I)
```

Regras Principais do `abstract`

- Uma classe abstrata **não pode ser instanciada** (`new`).
- Um método abstrato **não tem corpo** (`{}`) e deve ser sobrescrito por subclasses concretas.
- Uma classe que contém um método abstrato **deve** ser declarada como `abstract`.
- Uma subclasse pode ser **abstrata** se não implementar todos os métodos abstratos de sua superclasse. Nesse caso, ela também não pode ser instanciada.

Palavra chave “**final**”

Criando constantes e impedindo a herança e a sobrescrita

Palavra chave “**final**”

- Mesmo com a sobreposição de métodos e a herança sendo tão poderosas e úteis, em alguns casos pode ser desejável **evitar que ocorram**.
- Por exemplo, no nosso Sistema de Gestão de Biblioteca, na classe “Livro” podemos ter um método para calcular a “idade” do livro, ou seja, a quantos anos já fazem desde a publicação.
- Esse é um exemplo prático de método que NÃO deve ser sobrescrito, pois o cálculo não muda independente do tipo de livro.
- Neste exemplo, para evitar que alguém da equipe sobrescreva o método e possa, “sem querer ou não”, comprometer o cálculo, podemos utilizar a palavra chave “**final**”.

Palavra chave “**final**”

- Nas imagens podemos constatar que ao tentar sobrescrever um método definido com “final” na sua assinatura, temos um erro em tempo de compilação.

```
public abstract class Livro {

    public final int calcularTempoPublicacao() {
        int anoAtual = LocalDate.now().getYear();
        return anoAtual - this.getAnoPublicacao();
    }
```

```
public class LivroFisico extends Livro {

    public int calcularTempoPublicacao() {
        int ano
        return
    }

Cannot override the final method from Livro Java(67109265)
Go to Super Implementation
int LivroFisico.calcularTempoPublicacao()
```

Palavra chave “**final**”

- Quando queremos que uma classe não seja instanciada e sirva apenas para ser herdada utilizamos a palavra chave “**abstract**”.
- Com a palavra chave “**final**” temos o contrário, podemos definir que uma classe não possa ser herdada.
- Como era de se esperar, é inválido declarar uma classe como “**abstract**” e “**final**”.

Palavra chave “**final**”

```
public final class LivroFisico extends Livro {
    private int nExemplares;
    private String dimensoes;
```

```
public class SubLivroFisico extends LivroFisico{

}
```

The type SubLivroFisico cannot subclass the final class LivroFisico

LivroFisico

Palavra chave “**final**”

- Você também pode usar a palavra-chave “**final**” para definir **constantes** dentro do seu sistema. Isso é útil para valores que **não devem ser modificados após sua inicialização**.

```
private final int ANO_PUBLICACAO_MINIMO = 1400;
```

Resumo dos Usos

Onde Usar	Efeito
Variável	Torna a variável uma **constante**. O valor não pode ser alterado.
Método	Impede que o método seja **sobrescrito**.
Classe	Impede que a classe seja **herdada**.

Modificador **Static** em Java

Entendendo membros e métodos estáticos

O que é static?

- É um modificador usado para indicar que um membro pertence à classe, e não à instância.
- Neste caso, esse membro existe apenas uma vez na memória, independente do número de objetos instanciados.
- Principais usos:
  - Variáveis (atributos)
  - Métodos

Variáveis Estáticas

```java
public class Livro {
    public static int contadorLivros = 0;

    private String titulo;
    private String autor;
    private int anoPublicacao;

    public Livro(String titulo, String autor, int anoPublicacao) {
        this.titulo = titulo;
        this.autor = autor;
        this.anoPublicacao = anoPublicacao;
        contadorLivros++;
    }
```

- Neste exemplo, podemos ter uma variável que é incrementada toda vez que um objeto é instanciado.

```java
IO.println("Número de livros criados: %d".formatted(Livro.contadorLivros));
```

- Para acessar um membro de classe, utilize preferencialmente o nome da Classe.

Métodos Estáticos

- Métodos estáticos **não** acessam atributos ou métodos de instância diretamente
- Podem ser chamados sem criar um objeto
- Para chamar o método, podemos **e devemos** usar o nome da Classe

```java
public class Input {
    public static int scanInt(String message) {
        while (true) {
            try {
                int value = Integer.valueOf(IO.readln(message));
            } catch (Exception e) {
                IO.println("Valor Inválido! Digite um número inteiro.");
            }
        }
    }
}
```

```java
int anoPublicacao = Input.scanInt("Digite o ano de publicação: ");
```

Boas Práticas

- Use **static** para constantes e utilitários
- Evite acessar variáveis estáticas com objetos (use o nome da classe)
- Cuidado com estados globais → pode gerar dependências ruins